Пошаговый разбор алгоритма

Блочное LU-разложение · Алгоритм 37 пособия Головашкина

$$A' = A_{22} - L_{21}\,U_{12}$$

Пример 6 × 6 · четыре шага · зависимости · OpenMP · MPI

Intel Core i5-1135G7 · MinGW GCC 9.2 · MS-MPI

Содержание

Пошаговый разбор алгоритма

1. Что считает алгоритм
2. Блоки и четыре шага
3. Пример: матрица 6 × 6
4. Где работа и что от чего зависит
5. OpenMP: потоки делят задачи каждого шага
6. MPI: процессы делят матрицу по блочным строкам
7. Где это в коде
8. Вопросы для самопроверки

Пошаговый разбор алгоритма

Этот документ объясняет блочное LU-разложение (Алгоритм 37 из пособия Головашкина) и его параллельные версии на одном маленьком примере, который можно пересчитать на бумаге. Сначала — что и зачем считается, затем — весь алгоритм на матрице 6×6 с подробной арифметикой, и наконец — как ту же работу делят между собой потоки OpenMP и процессы MPI.

Как читать формулы и матрицы:

- индексы начинаются **с нуля**, как в программах: строки и столбцы $0, 1, \ldots, n-1$;
- α — размер малого блока, $N = n/\alpha$ — число блоков по одной стороне;
- A_{ij} — **малый блок** в блочной строке i и блочном столбце j (а не отдельный элемент); отдельный элемент обозначается a_{sq};
- в матрицах примера вертикальные и горизонтальные линии — границы малых блоков, **красным** выделены элементы, изменённые на этом шаге, **серым** — элементы, которые уже окончательны и больше не меняются.

1. Что считает алгоритм

1.1. LU-разложение

Нужно представить квадратную матрицу в виде произведения

$$A = L\,U,$$

где L — нижняя треугольная матрица с единицами на диагонали, U — верхняя треугольная. Например, для матрицы 2×2:

$$\begin{pmatrix} 2 & 1 \\ 4 & 5 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}.$$

Проверка: первая строка произведения — $1 \cdot (2,1) = (2,1)$, вторая — $2 \cdot (2,1) + 1 \cdot (0,3) = (4,5)$.

Зачем это нужно: систему $Ax = b$ после разложения решают две простые подстановки — прямая $Ly = b$ (сверху вниз) и обратная $Ux = y$ (снизу вверх). Разложение стоит $\frac{2}{3}n^3$ операций, каждая подстановка — всего n^2, поэтому основная работа — именно разложение.

1.2. Хранение на месте

Отдельные массивы для L и U не нужны. Единицы на диагонали L известны заранее и не хранятся, а остальные элементы L и U не пересекаются. Поэтому оба множителя записываются поверх A: строго под диагональю — элементы L, на диагонали и выше — элементы U. Для примера выше:

$$\begin{pmatrix} 2 & 1 \\ 4 & 5 \end{pmatrix} \longrightarrow \begin{pmatrix} 2 & 1 \\ \mathbf{2} & 3 \end{pmatrix}, \qquad \ell_{10} = 2,\ u_{00} = 2,\ u_{01} = 1,\ u_{11} = 3.$$

1.3. Метод Гаусса — то же самое

Обычный метод Гаусса уже строит LU-разложение. Чтобы занулить $a_{10} = 4$ под ведущим элементом $a_{00} = 2$, из второй строки вычитают первую, умноженную на $\ell_{10} = 4/2 = 2$:

$$(4,\ 5) - 2 \cdot (2,\ 1) = (0,\ 3).$$

Множитель 2 — это и есть элемент L, а то, что осталось от строк, — матрица U. Блочный алгоритм выполняет те же вычитания, но в другом порядке: группами по блокам.

2. Блоки и четыре шага

2.1. Разбиение на блоки

Матрица делится на $N \times N$ малых блоков размера $\alpha \times \alpha$. В примере $n = 6, \alpha = 2, N = 3$:

$$A = \left(\begin{array}{c|c|c} A_{00} & A_{01} & A_{02} \\ \hline A_{10} & A_{11} & A_{12} \\ \hline A_{20} & A_{21} & A_{22} \end{array}\right), \qquad A_{ij} = \text{строки } 2i, 2i{+}1;\ \text{столбцы } 2j, 2j{+}1.$$

В программах $\alpha = 32$: три блока 32×32 занимают 24 КБ и помещаются в самый быстрый кэш процессора (L1, 48 КБ). Основная работа идёт с блоками, которые уже лежат в кэше, — в этом смысл блочного алгоритма.

2.2. Откуда берутся шаги

Выделим первый блок и запишем разложение в три сомножителя (формула (18) пособия):

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11} & 0 \\ L_{21} & E \end{pmatrix} \begin{pmatrix} E & 0 \\ 0 & A' \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ 0 & E \end{pmatrix}.$$

Здесь (только в этой формуле, как в пособии) A_{11} — один малый блок в левом верхнем углу, A_{21} — блочный столбец под ним, A_{12} — блочная строка справа от него, A_{22} — всё остальное, E — единичные матрицы. Перемножим правую часть:

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11}U_{11} & L_{11}U_{12} \\ L_{21}U_{11} & L_{21}U_{12} + A' \end{pmatrix}.$$

Приравниваем блоки — получаем четыре равенства, и каждое даёт один шаг:

Шаг	Равенство	Известно	Находим	Как
1	$A_{11} = L_{11}U_{11}$	A_{11}	L_{11}, U_{11}	обычное LU маленького блока
2	$A_{21} = L_{21}U_{11}$	A_{21}, U_{11}	L_{21}	треугольные системы вида $xU = b$
3	$A_{12} = L_{11}U_{12}$	A_{12}, L_{11}	U_{12}	треугольные системы вида $Lx = b$
4	$A_{22} = L_{21}U_{12} + A'$	всё, кроме A'	A'	$A' = A_{22} - L_{21}U_{12}$ — **основная операция**

После шага 4 известны первая блочная строка U и первый блочный столбец L, а A' — меньшая матрица, которую осталось разложить. Её раскладывают теми же четырьмя шагами, и так далее, пока не останется один блок. Номер очередного «первого блока» — это номер итерации $k = 0, 1, \ldots, N-1$.

Порядок шагов определяется тем, что каждому нужно: шагам 2 и 3 нужен результат шага 1, шагу 4 — результаты шагов 2 и 3. Шаги 2 и 3 друг от друга не зависят.

3. Пример: матрица 6 × 6

3.1. Исходные данные

Матрица построена как произведение заранее выбранных L и U с целыми элементами, поэтому все промежуточные числа целые, а ответ известен:

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 & 0 & 0 \\ -1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 2 & 1 & 0 & 0 \\ 0 & 2 & -1 & 1 & 1 & 0 \\ 1 & -1 & 1 & 0 & 2 & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 2 & 1 & -1 & 0 & 1 & 2 \\ 0 & 3 & 1 & 2 & 0 & -1 \\ 0 & 0 & 2 & 1 & -1 & 0 \\ 0 & 0 & 0 & 4 & 1 & 1 \\ 0 & 0 & 0 & 0 & 3 & -2 \\ 0 & 0 & 0 & 0 & 0 & 2 \end{pmatrix}.$$

Их произведение — матрица, которую будем раскладывать:

$$\left(\begin{array}{cc|cc|cc} 2 & 1 & -1 & 0 & 1 & 2 \\ 4 & 5 & -1 & 2 & 2 & 3 \\ \hline -2 & 2 & 4 & 3 & -2 & -3 \\ 2 & 1 & 3 & 6 & 0 & 3 \\ \hline 0 & 6 & 0 & 7 & 5 & -3 \\ 2 & -2 & 0 & -1 & 6 & 1 \end{array}\right)$$

Задача алгоритма — получить из неё обратно L и U, записанные на месте A.

3.2. Итерация k = 0

Текущий блок — A_{00} (строки и столбцы 0, 1). Под ним блоки A_{10}, A_{20}, справа от него — A_{01}, A_{02}, остальные четыре блока — подматрица A_{22} из формулы раздела 2.2.

Шаг 1. LU блока A_{00}

$$A_{00} = \begin{pmatrix} 2 & 1 \\ 4 & 5 \end{pmatrix}.$$

Это разобранный в разделе 1 пример. Для каждой строки s ниже строки l блока: сначала множитель $\ell_{sl} = a_{sl}/u_{ll}$, затем из строки s вычитается строка l, умноженная на него. Здесь одна такая пара: $l = 0$, $s = 1$:

$$\ell_{10} = \frac{4}{2} = 2, \qquad a_{11} = 5 - 2 \cdot 1 = 3.$$

Итог: $L_{00} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}$, $U_{00} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$. Множитель 2 записан на место a_{10}:

$$\left(\begin{array}{cc|cc|cc} 2 & 1 & -1 & 0 & 1 & 2 \\ \mathbf{2} & \mathbf{3} & -1 & 2 & 2 & 3 \\ \hline -2 & 2 & 4 & 3 & -2 & -3 \\ 2 & 1 & 3 & 6 & 0 & 3 \\ \hline 0 & 6 & 0 & 7 & 5 & -3 \\ 2 & -2 & 0 & -1 & 6 & 1 \end{array}\right)$$

Шаг 2. Блоки L_{10} и L_{20}: системы $x\,U_{00} = b$

Нужно найти такие L_{10} и L_{20}, что $L_{10}U_{00} = A_{10}$ и $L_{20}U_{00} = A_{20}$. Каждая **строка** блока — отдельная система $x \cdot U_{00} = b$, где b — строка блока A, x — строка искомого блока L. Распишем её для U_{00}:

$$(x_0,\ x_1)\begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix} = (2x_0,\ x_0 + 3x_1) = (b_0,\ b_1).$$

Отсюда сначала $x_0 = b_0/2$, потом из b_1 вычитается вклад x_0: $b_1 \leftarrow b_1 - x_0 \cdot 1$, и $x_1 = b_1/3$. Посчитаем все четыре строки (строки 2, 3 — блок L_{10}; строки 4, 5 — блок L_{20}):

Строка	b	$x_0 = b_0/2$	$b_1 - x_0 \cdot 1$	$x_1 = (\ldots)/3$	Строка L
2	$(-2,\ 2)$	-1	$2 - (-1) = 3$	1	$(-1,\ 1)$
3	$(2,\ 1)$	1	$1 - 1 = 0$	0	$(1,\ 0)$
4	$(0,\ 6)$	0	$6 - 0 = 6$	2	$(0,\ 2)$
5	$(2,\ -2)$	1	$-2 - 1 = -3$	-1	$(1,\ -1)$

Проверка для строки 2: $(-1, 1) \cdot U_{00} = (-2,\ -1 + 3) = (-2,\ 2)$ — исходная строка.

Шаг 3. Блоки U_{01} и U_{02}: системы $L_{00}\,y = b$

Нужно найти U_{01} и U_{02} из $L_{00}U_{01} = A_{01}$ и $L_{00}U_{02} = A_{02}$. Теперь каждый **столбец** блока — система $L_{00}\,y = b$:

$$\begin{pmatrix}1 & 0\\ 2 & 1\end{pmatrix}\begin{pmatrix}y_0\\ y_1\end{pmatrix} = \begin{pmatrix}y_0\\ 2y_0 + y_1\end{pmatrix} = \begin{pmatrix}b_0\\ b_1\end{pmatrix} \quad\Longrightarrow\quad y_0 = b_0,\quad y_1 = b_1 - 2\,y_0.$$

Деления нет: на диагонали L единицы. Первая строка блочной строки не меняется, а из второй вычитается первая, умноженная на $\ell_{10} = 2$ (в программе это делается сразу для целой строки):

$$\text{строка 1, столбцы 2–5:}\quad (-1,\ 2,\ 2,\ 3) - 2\cdot(-1,\ 0,\ 1,\ 2) = (1,\ 2,\ 0,\ -1).$$

Итог: $U_{01} = \begin{pmatrix}-1 & 0\\ 1 & 2\end{pmatrix}$, $U_{02} = \begin{pmatrix}1 & 2\\ 0 & -1\end{pmatrix}$. Состояние после шагов 2 и 3:

$$\left(\begin{array}{cc|cc|cc}
2 & 1 & -1 & 0 & 1 & 2\\
2 & 3 & \mathbf{1} & \mathbf{2} & \mathbf{0} & \mathbf{-1}\\
\hline
\mathbf{-1} & \mathbf{1} & 4 & 3 & -2 & -3\\
\mathbf{1} & \mathbf{0} & 3 & 6 & 0 & 3\\
\hline
\mathbf{0} & \mathbf{2} & 0 & 7 & 5 & -3\\
\mathbf{1} & \mathbf{-1} & 0 & -1 & 6 & 1
\end{array}\right)$$

Первая блочная строка U и первый блочный столбец L готовы (верхние две строки и левые два столбца больше не изменятся).

Шаг 4. Основная операция $A'_{ij} = A_{ij} - L_{i0}\,U_{0j}$

Для каждой из четырёх пар блоков (i, j), $i, j \in \{1, 2\}$, из блока A_{ij} вычитается произведение двух уже найденных блоков: L_{i0} (из блочного столбца) и U_{0j} (из блочной строки).

Пара (1, 1).

$$L_{10}U_{01} = \begin{pmatrix}-1 & 1\\ 1 & 0\end{pmatrix}\begin{pmatrix}-1 & 0\\ 1 & 2\end{pmatrix} = \begin{pmatrix}1+1 & 0+2\\ -1+0 & 0+0\end{pmatrix} = \begin{pmatrix}2 & 2\\ -1 & 0\end{pmatrix},\qquad A'_{11} = \begin{pmatrix}4 & 3\\ 3 & 6\end{pmatrix} - \begin{pmatrix}2 & 2\\ -1 & 0\end{pmatrix} = \begin{pmatrix}2 & 1\\ 4 & 6\end{pmatrix}.$$

Пара (1, 2).

$$L_{10}U_{02} = \begin{pmatrix}-1 & 1\\ 1 & 0\end{pmatrix}\begin{pmatrix}1 & 2\\ 0 & -1\end{pmatrix} = \begin{pmatrix}-1 & -3\\ 1 & 2\end{pmatrix},\qquad A'_{12} = \begin{pmatrix}-2 & -3\\ 0 & 3\end{pmatrix} - \begin{pmatrix}-1 & -3\\ 1 & 2\end{pmatrix} = \begin{pmatrix}-1 & 0\\ -1 & 1\end{pmatrix}.$$

Пара (2, 1).

$$L_{20}U_{01} = \begin{pmatrix}0 & 2\\ 1 & -1\end{pmatrix}\begin{pmatrix}-1 & 0\\ 1 & 2\end{pmatrix} = \begin{pmatrix}2 & 4\\ -2 & -2\end{pmatrix},\qquad A'_{21} = \begin{pmatrix}0 & 7\\ 0 & -1\end{pmatrix} - \begin{pmatrix}2 & 4\\ -2 & -2\end{pmatrix} = \begin{pmatrix}-2 & 3\\ 2 & 1\end{pmatrix}.$$

Пара (2, 2).

$$L_{20}U_{02} = \begin{pmatrix}0 & 2\\ 1 & -1\end{pmatrix}\begin{pmatrix}1 & 2\\ 0 & -1\end{pmatrix} = \begin{pmatrix}0 & -2\\ 1 & 3\end{pmatrix},\qquad A'_{22} = \begin{pmatrix}5 & -3\\ 6 & 1\end{pmatrix} - \begin{pmatrix}0 & -2\\ 1 & 3\end{pmatrix} = \begin{pmatrix}5 & -1\\ 5 & -2\end{pmatrix}.$$

Каждая пара пишет только в свой блок и читает только L_{i0}, U_{0j} и свой блок — четыре пары можно считать в любом порядке и одновременно. Состояние после итерации $k = 0$:

$$\left(\begin{array}{cc|cc|cc}
2 & 1 & -1 & 0 & 1 & 2\\
2 & 3 & 1 & 2 & 0 & -1\\
\hline
-1 & 1 & \mathbf{2} & \mathbf{1} & \mathbf{-1} & \mathbf{0}\\
1 & 0 & \mathbf{4} & \mathbf{6} & \mathbf{-1} & \mathbf{1}\\
\hline
0 & 2 & \mathbf{-2} & \mathbf{3} & \mathbf{5} & \mathbf{-1}\\
1 & -1 & \mathbf{2} & \mathbf{1} & \mathbf{5} & \mathbf{-2}
\end{array}\right)$$

Нижний правый угол 4×4 — это A', дополнение Шура блока A_{00}. Он раскладывается дальше теми же шагами.

Сверка с формулой (18) и рисунком 8а пособия. Рисунок 8а показывает, как хранится матрица после одной итерации: в левом верхнем углу $L_{11}\backslash U_{11}$, справа от него — блочная строка U, под ним — L_{21}, в правом нижнем углу — A'. Это ровно та матрица, что выше (на рисунке над правым верхним блоком подписано U_{11}, но по тексту и формуле (18) там U_{12} — первая блочная строка U). Разнесём её по трём сомножителям формулы (18):

$$\underbrace{\left(\begin{array}{cc|cccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 & 0 & 0 \\ \hline -1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 2 & 0 & 0 & 1 & 0 \\ 1 & -1 & 0 & 0 & 0 & 1 \end{array}\right)}_{\begin{pmatrix} L_{11} & \\ L_{21} & E \end{pmatrix}} \underbrace{\left(\begin{array}{cc|cccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 2 & 1 & -1 & 0 \\ 0 & 0 & 4 & 6 & -1 & 1 \\ 0 & 0 & -2 & 3 & 5 & -1 \\ 0 & 0 & 2 & 1 & 5 & -2 \end{array}\right)}_{\begin{pmatrix} E & \\ & A' \end{pmatrix}} \underbrace{\left(\begin{array}{cc|cccc} 2 & 1 & -1 & 0 & 1 & 2 \\ 0 & 3 & 1 & 2 & 0 & -1 \\ \hline 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{array}\right)}_{\begin{pmatrix} U_{11} & U_{12} \\ & E \end{pmatrix}}$$

Произведение этих трёх матриц равно исходной A. Проверим один элемент, $a_{22} = 4$: строка 2 произведения первых двух сомножителей — $(-1,\ 1,\ 2,\ 1,\ -1,\ 0)$ (первые два числа — из L_{21}, остальные — строка 0 матрицы A'), столбец 2 третьего сомножителя — $(-1,\ 1,\ 1,\ 0,\ 0,\ 0)^T$:

$$(-1)(-1) + 1 \cdot 1 + 2 \cdot 1 = 4 = a_{22}.$$

Так же формула (18) выполняется после каждой следующей итерации: роль L_{11}, U_{11} играют уже все готовые строки и столбцы, роль A' — оставшийся угол. Это проверено скриптом на всех трёх итерациях примера в точной арифметике и на случайной матрице 64×64 (отклонение произведения от A — не больше 10^{-14}, это ошибка округления).

Рисунки 8б и 8в относятся к двум упрощённым вариантам алгоритма (с. 61 пособия), где первые шаги заменены на α итераций LU через gaxpy или через мм.; в программах они не используются.

Как программа считает произведение блоков. Не через формулу «строка на столбец», а строками: для каждой строки s блока A_{ij} и каждого l из текущего блока из строки s вычитается строка l блока U_{0j}, умноженная на ℓ_{sl}. Для пары (1, 1), строка 2:

$$(4,\ 3) - \underbrace{(-1)}_{\ell_{20}} \cdot (-1,\ 0) - \underbrace{1}_{\ell_{21}} \cdot (1,\ 2) = (4,\ 3) - (1,\ 0) - (1,\ 2) = (2,\ 1).$$

Результат тот же, но каждая операция — «строка минус число на строку» (saxpy) над соседними элементами памяти. Именно эту операцию компилятор векторизует на этапе 2.

3.3. Итерация k = 1

Теперь текущий блок — A_{11}, а раскладывается подматрица A' из строк и столбцов 2–5.

Шаг 1. $A_{11} = \begin{pmatrix} 2 & 1 \\ 4 & 6 \end{pmatrix}$: $\ell_{32} = 4/2 = 2,\ a_{33} = 6 - 2 \cdot 1 = 4$. Итог: $L_{11} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}, U_{11} = \begin{pmatrix} 2 & 1 \\ 0 & 4 \end{pmatrix}$.

Шаг 2. Один блок A_{21} (строки 4, 5, столбцы 2, 3); система $x\,U_{11} = b$ даёт $x_0 = b_0/2, x_1 = (b_1 - x_0)/4$:

Строка	b	x_0	$b_1 - x_0$	x_1	Строка L
4	$(-2,\ 3)$	-1	4	1	$(-1,\ 1)$
5	$(2,\ 1)$	1	0	0	$(1,\ 0)$

Шаг 3. Один блок A_{12} (строки 2, 3, столбцы 4, 5): строка 2 не меняется, из строки 3 вычитается строка 2, умноженная на $\ell_{32} = 2$: $(-1,\ 1) - 2 \cdot (-1,\ 0) = (1,\ 1)$.

$$\left(\begin{array}{cc|cc|cc} 2 & 1 & -1 & 0 & 1 & 2 \\ 2 & 3 & 1 & 2 & 0 & -1 \\ \hline -1 & 1 & 2 & 1 & -1 & 0 \\ 1 & 0 & 2 & 4 & \mathbf{1} & \mathbf{1} \\ \hline 0 & 2 & \mathbf{-1} & \mathbf{1} & 5 & -1 \\ 1 & -1 & \mathbf{1} & \mathbf{0} & 5 & -2 \end{array}\right)$$

Шаг 4. Одна пара (2, 2):

$$L_{21}U_{12} = \begin{pmatrix} -1 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} -1 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ -1 & 0 \end{pmatrix}, \qquad A'_{22} = \begin{pmatrix} 5 & -1 \\ 5 & -2 \end{pmatrix} - \begin{pmatrix} 2 & 1 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 3 & -2 \\ 6 & -2 \end{pmatrix}.$$

$$\left(\begin{array}{cc|cc|cc} 2 & 1 & -1 & 0 & 1 & 2 \\ 2 & 3 & 1 & 2 & 0 & -1 \\ \hline -1 & 1 & 2 & 1 & -1 & 0 \\ 1 & 0 & 2 & 4 & 1 & 1 \\ \hline 0 & 2 & -1 & 1 & \mathbf{3} & \mathbf{-2} \\ 1 & -1 & 1 & 0 & \mathbf{6} & \mathbf{-2} \end{array}\right)$$

3.4. Итерация k = 2

Остался последний блок $A_{22} = \begin{pmatrix} 3 & -2 \\ 6 & -2 \end{pmatrix}$. Ниже и правее блоков нет, поэтому шаги 2–4 пусты, выполняется только шаг 1:

$$\ell_{54} = \frac{6}{3} = 2, \qquad a_{55} = -2 - 2 \cdot (-2) = 2.$$

3.5. Результат

$$\left(\begin{array}{cc|cc|cc} 2 & 1 & -1 & 0 & 1 & 2 \\ 2 & 3 & 1 & 2 & 0 & -1 \\ \hline -1 & 1 & 2 & 1 & -1 & 0 \\ 1 & 0 & 2 & 4 & 1 & 1 \\ \hline 0 & 2 & -1 & 1 & 3 & -2 \\ 1 & -1 & 1 & 0 & 2 & 2 \end{array}\right)$$

Под диагональю записана ровно матрица L, на диагонали и выше — ровно U из раздела 3.1.

3.6. Решение системы

Возьмём b — суммы строк исходной матрицы: $b = (5, 15, 2, 15, 15, 6)$. Тогда точное решение — $x^* = (1, \dots, 1)$, так программы проверяют свою точность.

Прямая подстановка $Ly = b$ — сверху вниз, $y_i = b_i - \sum_{j<i} \ell_{ij} y_j$:

$$\begin{aligned} y_0 &= 5, \\ y_1 &= 15 - 2 \cdot 5 = 5, \\ y_2 &= 2 - \big((-1) \cdot 5 + 1 \cdot 5\big) = 2, \\ y_3 &= 15 - (1 \cdot 5 + 0 \cdot 5 + 2 \cdot 2) = 6, \\ y_4 &= 15 - \big(0 \cdot 5 + 2 \cdot 5 + (-1) \cdot 2 + 1 \cdot 6\big) = 1, \\ y_5 &= 6 - \big(1 \cdot 5 + (-1) \cdot 5 + 1 \cdot 2 + 0 \cdot 6 + 2 \cdot 1\big) = 2. \end{aligned}$$

Обратная подстановка $Ux = y$ — снизу вверх, $x_i = \big(y_i - \sum_{j>i} u_{ij} x_j\big) / u_{ii}$:

$$
\begin{aligned}
x_5 &= 2/2 = 1,\\
x_4 &= \bigl(1 - (-2)\cdot 1\bigr)/3 = 1,\\
x_3 &= (6 - 1 - 1)/4 = 1,\\
x_2 &= (2 - 1 + 1 - 0)/2 = 1,\\
x_1 &= (5 - 1 - 2 - 0 + 1)/3 = 1,\\
x_0 &= (5 - 1 + 1 - 0 - 1 - 2)/2 = 1.
\end{aligned}
$$

В точной арифметике $x = x^*$. В программах числа с плавающей точкой, поэтому x отличается от единиц на величину порядка 10^{-15} — это и есть измеряемая погрешность `err`.

3.7. Если n не делится на α

При $n = 5$ и $\alpha = 2$ блоков $N = \lceil 5/2 \rceil = 3$, и последний блок — размера 1×1 (строка и столбец 4). Все шаги те же, только границы последнего блока короче: в программах это делает макрос `END(i)`, возвращающий $\min((i+1)\alpha,\, n)$.

4. Где работа и что от чего зависит

4.1. Сколько стоит каждый шаг

Посчитаем арифметические операции в примере: каждое деление — одна операция, каждое «$a \leftarrow a - \ell u$» — две (умножение и вычитание):

Итерация	Шаг 1	Шаг 2	Шаг 3	Шаг 4
$k = 0$	3	16	8	64
$k = 1$	3	8	4	16
$k = 2$	3	—	—	—
всего	9	24	12	80

Уже на матрице 6×6 шаг 4 — 80 операций из 125, то есть 64 % работы. В общем случае на шаг 4 приходится $\approx \frac{2}{3}n^3$ операций из $\frac{2}{3}n^3$, а шаги 1–3 растут лишь как $n^2\alpha$: при $n = 3000$ шаг 4 — 98,4 % всей работы. **Поэтому ускорять и распараллеливать нужно прежде всего шаг 4.**

4.2. Граф зависимостей одной итерации

```
                  шаг 1: LU блока A_kk          (1 задача, внутри последовательно)
                 /                     \
шаг 2: L_ik, i > k                   шаг 3: U_kj, j > k
(N-k-1 независимых задач)            (N-k-1 независимых задач)
                 \                     /
              шаг 4: A'_ij = A_ij - L_ik U_kj, i, j > k
              ((N-k-1)^2 независимых задач)
                            |
                  итерация k + 1
```

- **Шаг 1 внутри последователен**: строка s блока использует строки $l < s$, уже обработанные. Но его работа ничтожна (0,01 %).
- **Шаг 2**: блоки L_{ik} для разных i не зависят друг от друга — каждый читает только U_{kk} и пишет только свои строки. В примере при $k = 0$ строки 2–3 и 4–5 можно считать одновременно.
- **Шаг 3**: блоки U_{kj} для разных j независимы — каждый читает L_{kk} и пишет только свои столбцы.

- **Шаг 4**: пары (i, j) независимы — пара пишет только в блок A_{ij}, а читает L_{ik} и U_{kj}, в которые на этом шаге никто не пишет.
- **Итерации по k последовательны**: итерация $k+1$ раскладывает матрицу A', которую построила итерация k. В примере блок A_{11} на итерации 1 — это $\begin{pmatrix} 2 & 1 \\ 4 & 6 \end{pmatrix}$, он существует только после шага 4 итерации 0.

Число независимых задач в примере ($N = 3$) и в реальной задаче ($n = 3000, \alpha = 32, N = 94$):

	Шаг 2	Шаг 3	Шаг 4
пример, $k = 0$	2	2	4
пример, $k = 1$	1	1	1
$n = 3000, k = 0$	93	93	8649
$n = 3000, k = 90$	3	3	9

Задач на шаге 4 много почти до самого конца разложения, поэтому алгоритм хорошо параллелится.

4.3. Что пошло бы не так без соблюдения зависимостей

Допустим, шаг 4 итерации 0 начат раньше, чем закончен шаг 2: пара (1, 1) взяла блок L_{10}, где ещё лежат исходные числа $(-2, 2)$ и $(2, 1)$ вместо $(-1, 1)$ и $(1, 0)$. Тогда

$$\begin{pmatrix} -2 & 2 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} -1 & 0 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 4 \\ -1 & 2 \end{pmatrix}, \qquad \begin{pmatrix} 4 & 3 \\ 3 & 6 \end{pmatrix} - \begin{pmatrix} 4 & 4 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 4 & 4 \end{pmatrix}$$

вместо правильного $\begin{pmatrix} 2 & 1 \\ 4 & 6 \end{pmatrix}$, и на следующей итерации ведущий элемент окажется нулём — деление на ноль.

Поэтому между шагами нужна синхронизация: в OpenMP её даёт неявный барьер в конце каждого параллельного цикла, в MPI — рассылка блочной строки.

5. OpenMP: потоки делят задачи каждого шага

5.1. Как устроена программа

Все потоки работают с одной общей матрицей в памяти. Код последовательной программы остаётся прежним, только перед циклами шагов 2–4 стоит директива:

```
for (int k = 0; k < N; k++) {
    /* шаг 1 — последовательно */

    #pragma omp parallel for schedule(static)              /* шаг 2 */
    for (int i = k + 1; i < N; i++) { ... }

    #pragma omp parallel for schedule(static)              /* шаг 3 */
    for (int j = k + 1; j < N; j++) { ... }

    #pragma omp parallel for collapse(2) schedule(static)  /* шаг 4 */
    for (int i = k + 1; i < N; i++)
        for (int j = k + 1; j < N; j++) { ... }
}
```

Что делает каждая директива:

1. главный поток доходит до цикла и будит команду из P потоков (fork);
2. библиотека OpenMP делит итерации цикла на P непрерывных кусков — это `schedule(static)`;
3. каждый поток выполняет свой кусок;

4. в конце цикла все ждут друг друга (неявный барьер, join), и только потом главный поток идёт дальше.

`collapse(2)` склеивает два вложенных цикла шага 4 в один: пары (i, j) нумеруются подряд и делятся между потоками как один длинный список.

5.2. Пример: P = 2 потока, итерация k = 0

Шаг 1 — только главный поток (поток 0) раскладывает A_{00}; поток 1 в это время не занят.

Шаг 2 — итерации $i = 1, 2$ делятся пополам:

Поток	Итерации	Что считает	Строки матрицы
0	$i = 1$	L_{10}	2, 3
1	$i = 2$	L_{20}	4, 5

Шаг 3 — итерации $j = 1, 2$ делятся так же:

Поток	Итерации	Что считает	Столбцы матрицы
0	$j = 1$	U_{01}	2, 3
1	$j = 2$	U_{02}	4, 5

Шаг 4 — четыре пары нумеруются $0, 1, 2, 3$ и делятся на два куска по две:

Номер	Пара (i, j)	Поток	Блок, в который пишет
0	(1, 1)	0	строки 2–3, столбцы 2–3
1	(1, 2)	0	строки 2–3, столбцы 4–5
2	(2, 1)	1	строки 4–5, столбцы 2–3
3	(2, 2)	1	строки 4–5, столбцы 4–5

Временная диаграмма итерации:

```
поток 0: [шаг 1 A00] [L10] | [U01] | [пара 1,1][пара 1,2] | → итерация k=1
поток 1:   (ждёт)    [L20] | [U02] | [пара 2,1][пара 2,2] |
                           ^       ^                      ^
                      барьер   барьер                барьер
```

На итерации $k = 1$ в каждом шаге всего одна задача, и второй поток простаивает. На большой матрице такие «хвосты» — малая доля работы: при $n = 3000$ и $P = 4$ на шаге 4 итерации $k = 0$ каждому потоку достаётся около 2160 пар блоков.

5.3. Почему результат совпадает с последовательным

Каждый элемент матрицы считает ровно один поток, и в том же порядке операций: те же вычитания, с теми же множителями, по возрастанию l. Меняется только то, *кто* считает, а не *как*. Поэтому OpenMP-версия даёт тот же результат до последнего бита при любом числе потоков.

5.4. Чего ждать по скорости

Последовательная часть — только шаг 1, доля α^2/n^2 (при $n = 3000$ — 10^{-4}), поэтому по закону Амдала ускорение должно быть близко к P. На ноутбуке получилось 2,2–2,4 на четырёх потоках: при загрузке всех ядер процессор снижает частоту, и это подтвердил контрольный опыт с независимыми копиями программы (см. отчёт по лабораторной работе 3).

6. MPI: процессы делят матрицу по блочным строкам

6.1. Отличие от OpenMP

Процессы MPI **не видят памяти друг друга**: каждый хранит свой кусок матрицы, а данными обмениваются только сообщениями. Поэтому сначала нужно решить, кто что хранит.

Посмотрим, что нужно шагам на итерации k:

- шаги 1 и 3 работают только с **блочной строкой** k (блоки A_{kk} и A_{kj});
- шаги 2 и 4 для блока i пишут только в **блочную строку** i, а читают из чужого лишь блочную строку k (U_{kk} для шага 2 и U_{kj} для шага 4).

Отсюда схема: матрица делится между процессами **по блочным строкам**. Владелец строки k выполняет шаги 1 и 3 и рассылает готовую блочную строку всем, а дальше каждый процесс выполняет шаги 2 и 4 над своими строками. Это единственный обмен за итерацию.

6.2. Циклическое распределение

Блочная строка i принадлежит процессу $i \bmod p$. Для примера и $p = 2$:

Блочная строка	Строки матрицы	Владелец
0	0, 1	процесс 0
1	2, 3	процесс 1
2	4, 5	процесс 0

Свои строки процесс хранит подряд в локальном массиве. У процесса 0 это строки 0, 1, 4, 5 — его блок 2 лежит на локальных строках 2–3; у процесса 1 — строки 2, 3 на локальных строках 0–1. Смещение считает макрос `LOCAL_ROW(i)` = $\lfloor i/p \rfloor \cdot \alpha$ строк.

Почему циклически, а не первые строки одному процессу, последние — другому? На итерации k работа есть только у блоков **ниже** k. При $N = 94$ и $p = 4$ с распределением кусками по 24 блока процесс 0 после 24-й итерации остался бы без работы, а процесс 3 работал бы до конца. При циклическом распределении у всех процессов почти до конца поровну блоков ниже k.

6.3. Пример: p = 2 процесса

Итерация k = 0. Владелец блочной строки 0 — процесс 0.

1. Процесс 0 выполняет шаг 1 (LU блока A_{00}) и шаг 3 (U_{01}, U_{02}) — это те же вычисления, что в разделе 3.2, только над его строками 0 и 1.
2. Процесс 0 копирует готовую блочную строку в буфер `W` — 2 строки по 6 чисел:

$$W = \begin{pmatrix} 2 & 1 & -1 & 0 & 1 & 2 \\ 2 & 3 & 1 & 2 & 0 & -1 \end{pmatrix} \qquad \big(\underbrace{L_{00}\backslash U_{00}}_{\text{столбцы 0, 1}} \mid \underbrace{U_{01}}_{2,3} \mid \underbrace{U_{02}}_{4,5} \big).$$

3. `MPI_Bcast` рассылает эти 12 чисел процессу 1.
4. Каждый процесс обрабатывает **свои** блоки ниже $k = 0$. Процессу 1 достаётся блок 1 (строки 2, 3): шаг 2 даёт L_{10}, шаг 4 — пары (1, 1) и (1, 2). Процессу 0 — блок 2 (строки 4, 5): шаг 2 даёт L_{20}, шаг 4 — пары (2, 1) и (2, 2). Числа — ровно те, что в разделе 3.2: процессу нужны только его строки и буфер `W` (U_{00} для шага 2, U_{01} и U_{02} для шага 4).

Итерация k = 1. Владелец блочной строки 1 — процесс 1.

1. Процесс 1 выполняет шаг 1 для блока A_{11} и шаг 3 для U_{12} над своими строками 2, 3.

2. Буфер — 2 строки по 4 числа (начиная с диагонального блока, столбцы 2–5):

$$W = \begin{pmatrix} 2 & 1 & -1 & 0 \\ 2 & 4 & 1 & 1 \end{pmatrix}.$$

3. `MPI_Bcast` от процесса 1 процессу 0 — 8 чисел.

4. Блок ниже $k = 1$ один — блок 2, он у процесса 0: шаг 2 даёт $L_{21} = \begin{pmatrix} -1 & 1 \\ 1 & 0 \end{pmatrix}$, шаг 4 — новый $A_{22} = \begin{pmatrix} 3 & -2 \\ 6 & -2 \end{pmatrix}$. Процесс 1 своих блоков ниже не имеет и ждёт.

Итерация k = 2. Владелец — процесс 0: шаг 1 для последнего блока. Рассылать незачем — ниже блоков нет.

Временная диаграмма:



В каждой итерации сначала работает только владелец блочной строки (шаги 1 и 3), затем рассылка, затем все параллельно выполняют шаги 2 и 4 над своими строками.

Всего передано $12 + 8 = 20$ чисел. В общем случае на итерации k рассылается $\alpha\,(n - k\alpha)$ чисел, за всё разложение — около $n^2/2$; при $n = 3000$ это 36 МБ.

После разложения процесс 0 собирает все строки, решает систему и считает погрешность. Раздача строк перед замером и сбор после него во время не входят.

6.4. Чем MPI-версия отличается от OpenMP на том же примере

	OpenMP, $P = 2$	MPI, $p = 2$
память	одна матрица на всех	у каждого свои строки
кто делает шаг 1	главный поток	владелец блочной строки k
шаг 2	потоки делят блоки	каждый — свои блоки
шаг 3	потоки делят блоки	только владелец
шаг 4, $k = 0$	поток 0: (1,1), (1,2); поток 1: (2,1), (2,2)	процесс 1: (1,1), (1,2); процесс 0: (2,1), (2,2)
синхронизация	барьер после каждого шага	`MPI_Bcast` блочной строки
обмен данными	не нужен	20 чисел за разложение

В MPI шаг 3 выполняет один процесс, пока остальные ждут рассылку, — поэтому MPI-версия немного медленнее OpenMP-версии. Зато матрица распределена: при $n = 3000$ и $p = 4$ каждый процесс хранит около 18 МБ вместо 72 МБ, и программа может работать на кластере, где у процессов нет общей памяти.

Результат, как и в OpenMP, совпадает с последовательным до последнего бита: каждый элемент считает один процесс в том же порядке, а рассылка копирует числа без изменений.

7. Где это в коде

Что	Этап 1	Этап 2	Этап 3	Этап 4
шаг 1	цикл `/* 1. ... */` в `block_lu`	то же через `saxpy`	то же, последовательно	у владельца, в `block_lu_mpi`
шаг 2	`/* 2. ... */`	через `saxpy`	`#pragma omp parallel for` по i	свои блоки после `MPI_Bcast`
шаг 3	`/* 3. ... */`	через `saxpy`	`#pragma omp parallel for` по j	у владельца
шаг 4	`/* 4. ... */`	через `saxpy`	`#pragma omp parallel for collapse(2)`	свои блоки после `MPI_Bcast`

Номера строк и построчный разбор — в справочнике по коду.

8. Вопросы для самопроверки

1. **Почему шаги 2 и 3 можно выполнять в любом порядке?** Шагу 2 нужен только U_{kk}, шагу 3 — только L_{kk}; оба получены на шаге 1, и шаги пишут в разные блоки.
2. **Почему в шаге 3 нет деления, а в шаге 2 есть?** В шаге 3 система с L, у которой единицы на диагонали; в шаге 2 — с U, у которой на диагонали ведущие элементы.
3. **Почему нельзя распараллелить цикл по k?** Итерация $k+1$ раскладывает матрицу A', построенную итерацией k (раздел 4.3).
4. **Какой шаг важнее всего ускорять и почему?** Шаг 4: на него приходится 98 % операций при $n = 3000$.
5. **Что будет, если в OpenMP-версии убрать барьер после шага 2?** Шаг 4 может прочитать незаконченный блок L_{ik} и получить неверный результат (раздел 4.3).
6. **Сколько чисел рассылается в MPI-версии на итерации k?** $\alpha\,(n - k\alpha)$: блочная строка k от диагонального блока до конца строки.
7. **Почему распределение строк в MPI циклическое?** Чтобы у всех процессов оставалась работа: на итерации k работают только блоки ниже k.
8. **Почему все версии дают одинаковую погрешность, а векторная — чуть другую?** Параллелизм меняет только исполнителя, а FMA в векторной версии вычисляет $a - \ell u$ с одним округлением вместо двух.
9. **Зачем вообще блоки, если поэлементный метод Гаусса делает то же самое?** Блочный алгоритм выполняет основную работу над тремя блоками, которые помещаются в кэш, и состоит из независимых задач — это и даёт возможность ускорения на этапах 2–4.